

Sprint 2 Demo Run Script

Dartly — ATS for Candidates | CS 490 Capstone

Quick Reference

Item	Value
App URL	http://localhost:3000
CI	.github/workflows/ci.yml
Test user A email	testuser@gmail.com
Test user A password	Testpass1!
Test user A ID	a818c364-412a-4545-8c88-f7b4cba05307
Test user B email	testuser2@gmail.com
Test user B password	Testpass1!
Test user B ID	77475f79-adb0-4de2-a61c-5ff34eb96ce7
Seed command	bun run db:clean && bun run db:seed
Test command	bun run test
CI pipeline	lint → type-check → test → build

Seeded Data Summary

- **88 jobs** across 7 stages (Interested, Applied, Interview, Offer, Rejected, Ghosted, Archived)
- User A: 42 jobs, User B: 43 jobs
- **2 profiles** with 3/2 experiences, 2/2 educations, 6/6 skills
- **Activities and stage history** auto-generated per job based on stage transitions
- **Dashboard metrics:** Total Jobs, Active, Response Rate, Interview Rate, Rejection Rate, Ghost Rate, Offer Count, pipeline distribution bar

Pre-Demo Checklist

- ☐ Run `bun dev` — confirm app loads at `http://localhost:3000`
- ☐ Run `bun run db:clean && bun run db:seed` — fresh seed data
- ☐ Login as User A (`testuser@gmail.com` / `Testpass1!`)
- ☐ Open CI tab: `https://github.com/justincordova/dartly/actions` (or repo URL)
- ☐ Open VS Code with these files ready to show:
 - `src/services/jobs.ts` (stage transition transaction)
 - `src/tests/services/jobs.test.ts` (stage transition tests)
 - `src/tests/api/jobs-id.test.ts` (auth/ownership tests)
 - `src/tests/services/profile.test.ts` (profile sync tests)
 - `src/tests/services/metrics.test.ts` (metrics computation tests)

Phase A — Scripted Product Demo (8 minutes)

A1. Dashboard Search/Filter/Sort and Card Status (2 min)

Step	Action	What to Show
1	Open Dashboard	42 seeded jobs in card grid (User A), metrics panel at top (Total Jobs, Active, Response Rate, Interview Rate, Rejection Rate, Ghost Rate) plus pipeline distribution bar with stage colors and counts
2	Search	Type "engineer" in search bar → results filter in real time (matches title, company, location, description, notes)
3	Filter	Set Stage filter to "Interview" → shows only interview-stage jobs
4	Sort	Change sort from "Most recent" to "Company A-Z" → cards reorder
5	Card indicators	Point out: color-coded stage badges (blue=Applied, yellow=Interview, green=Offer, red=Rejected, purple=Ghosted, orange=Archived, gray=Interested), urgency indicators (Overdue/Due soon), priority flags
6	Metrics panel	Point out the pipeline distribution bar showing proportional stage breakdown (hover/click to expand), Ghost Rate metric card, Interview Rate, Rejection Rate, and the offer count badge if offers exist

A2. Card Expansion to Job Detail and Workflow (2 min)

Step	Action	What to Show
1	Click a job card title	Navigates to /dashboard/{jobId} — Job Detail page loads
2	Show tab structure	Point out 5 tabs: Overview, Timeline, Interviews, Follow-ups, Documents
3	Click "Edit" on Overview	Form becomes editable — show all fields: title, company, stage, location, deadline, compensation, priority toggle, description, recruiter notes, personal notes
4	Change stage	Switch stage from "Applied" to "Interview" → click "Save"
5	Verify persistence	Stage badge updates, page reflects new stage. Click back to Dashboard → metrics updated

A3. Interview/Activity/Outcome Workflow (2 min)

Step	Action	What to Show
1	Click "Interviews" tab	Shows existing seeded interviews
2	Add interview	Click "Add interview" → fill round type (e.g., "Technical Screen"), date/time, notes → save

3	Click "Follow-ups" tab	Shows existing follow-ups with overdue highlighting
4	Add follow-up	Click "Add follow-up" → fill title, due date, notes → save
5	Click "Timeline" tab	Show unified timeline — all activity types visible (STAGE, INTERVIEW, FOLLOWUP, NOTE), auto-logged stage changes with "Stage changed: Applied → Interview"
6	Update outcome	Go back to Overview tab, change stage to "Ghosted" or "Rejected" → save → timeline auto-logs the change

A4. Profile Completion + AI Job-Context Document Flow (2 min)

Step	Action	What to Show
1	Navigate to /profile	Show 6 sections: Identity, Summary, Experience (3 entries), Education (2 entries), Skills (6 entries), Career Preferences (target roles, locations, work mode, salary)
2	Point out completion indicator	"X of 14 complete" tracker at top of profile
3	Navigate back to a Job Detail	Click "Documents" tab
4	Generate AI resume	Click "Generate Resume" → Gemini 2.5 Flash generates resume using profile + job context → auto-saved as document linked to job → redirects to document viewer
5	Generate AI cover letter	Go back to Job Detail → Documents tab → Click "Generate Cover Letter" → shows generated cover letter
6	Rewrite/improve	In document viewer, use Rewrite panel → type "Make it more concise" → shows side-by-side diff → click "Accept" to save as new version
7	Show dashboard metrics	Navigate to Dashboard → metrics panel shows all rates (Response, Interview, Rejection, Ghost), pipeline bar reflects stage distribution

Phase B — Technical Evidence and Q&A (7 minutes)

B1. Workflow and Data Integrity Evidence (2 min)

Demo: Stage change propagation (end-to-end)

Step	Action	What to Show
1	In the app	Change a job stage in Job Detail Overview → save
2	Check Timeline tab	Auto-created STAGE activity: "Stage changed: Applied → Interview"
3	Check Dashboard	lastActivityAt bumped → job surfaces at top with "Most recent" sort. Metrics panel reflects new stage counts and rates (Ghost Rate

		updates if moved to/from Ghosted)
4	Open src/services/jobs.ts	Show the Prisma transaction at lines 115–162 that atomically: (1) updates the Job record, (2) bumps lastActivityAt (line 137), (3) creates JobStageHistory (line 142), (4) creates JobActivity (line 152)
5	Show stage detection	Point to line 112 : const stageChanged = prismaStage && prismaStage !== existing.stage — gates history/activity creation
6	Show Ghosted handling	Point to src/services/metrics.ts — Ghosted excluded from active applications (line 25), ghost rate computed separately (line 54–56), response rate excludes GHOSTED transitions (line 37)

Show unit test:

Open `src/tests/services/jobs.test.ts` :

Test	Line	What it proves
"creates stage history and STAGE activity when stage changes"	100	Verifies jobStageHistory.create and jobActivity.create are called with correct from/to stages and activity metadata
"updates job without creating stage history when stage unchanged"	87	Negative case — verifies NO history/activity is created when stage stays the same
"returns null when job not found"	122	Edge case — updateJob returns null for nonexistent job

Show metrics test:

Open `src/tests/services/metrics.test.ts` :

Test	Line	What it proves
"computes active applications excluding Rejected, Archived, and Ghosted"	58	Verifies Ghosted jobs are excluded from active count
"computes ghost rate"	121	Verifies ghost rate is calculated as percentage of non-Interested jobs
"computes interview rate"	94	Verifies interview rate includes Interview + Offer stages
"computes rejection rate"	108	Verifies rejection rate as percentage of non-Interested jobs

B2. CI and Meaningful Testing Evidence (3 min)

CI pipeline — Open `.github/workflows/ci.yml` or GitHub Actions tab:

Runs on every PR/push to `main` :

- 1. bun install --frozen-lockfile
- 2. bun prisma generate
- 3. bun lint (Biome)
- 4. bun run type-check (tsc --noEmit)
- 5. bun run test (Vitest — 31 files, 265 tests)
- 6. bun run build (Next.js production build)

Show specific tests:

Requirement	File	Test Name	Line
Dashboard workflow	src/tests/services/jobs.test.ts	"creates stage history and STAGE activity when stage changes"	100
Profile completion	src/tests/services/profile.test.ts	"updates existing, creates new, and deletes removed experiences"	148
Metrics computation	src/tests/services/metrics.test.ts	"computes ghost rate"	121
Non-happy-path (401)	src/tests/api/jobs-id.test.ts	"returns 401 when not authenticated"	78
Non-happy-path (ownership)	src/tests/api/jobs-id.test.ts	"returns 404 when user does not own the job"	86

Explain the negative-path test:

The ownership denial test (jobs-id.test.ts:86) verifies that when user A tries to update user B's job, the service returns null and the API responds with 404 — not 403. This prevents cross-account data access without leaking which resources exist. The requireAuth() gate at the top of every handler plus userId scoping in service queries provides defense-in-depth.

Test suite summary: 31 files, 265 tests — covering dashboard workflow, profile sync, metrics computation, AI generation, document management, and negative-path (401/400/404/429/validation).

B3. Architecture and Implementation Q&A (2 min)

Prepare answers for these potential questions:

Q1: How does Sprint 2 extend Sprint 1 without breaking prior flows?

Sprint 1 established auth, job CRUD, and the dashboard shell. Sprint 2 added profile sub-models (Experience, Education, Skill with cascade deletes), activity/interview/follow-up polymorphic types on JobActivity , AI document generation pipeline (Gemini 2.5 Flash → Document → DocumentVersion → JobDocumentLink), and metrics computation. All new features are additive — no Sprint 1 endpoints were modified, only extended. The (app)/layout.tsx auth guard still protects all routes.

Q2: Where is stage transition logic implemented and why?

In src/services/jobs.ts:107–162 — inside the updateJob() service function, wrapped in a single Prisma \$transaction() . This ensures the job update, stage history record, activity log, and

`lastActivityAt` bump are atomic. If any step fails, the whole change rolls back. Putting it in the service layer (not the API route) means it's reusable and testable independently of HTTP concerns.

Q3: How does Job Detail coordinate timeline, interview, and document actions?

Job Detail (`/dashboard/[jobId]/page.tsx`) fetches the job and its activities via `GET /api/jobs/{id}` , then distributes data to tab components. The Timeline tab shows ALL activities (unfiltered). Interviews and Follow-ups tabs filter by `type === "INTERVIEW"` and `type === "FOLLOWUP"` respectively. All three use the same `POST /api/jobs/{id}/activities` endpoint — the `type` field on `JobActivity` is the polymorphic discriminator. Documents tab fetches via `GET /api/jobs/{id}/documents` and links to the document viewer.

Q4: How do profile entities feed AI generation inputs?

The AI resume/cover letter endpoints (`src/app/api/ai/resume/route.ts` , `cover-letter/route.ts`) call `getProfile(userId)` which fetches the profile with all relations (experiences, educations, skills). The `generateResumeDraft()` and `generateCoverLetterDraft()` functions in `src/services/ai.ts` serialize the full profile (name, contact, headline, summary, experiences with descriptions, education, skills with categories) plus job context (title, company, description) into a structured prompt for Gemini 2.5 Flash.

Q5: What technical debt did Sprint 2 introduce, and how will Sprint 3 address it?

- Metrics are computed on-the-fly (no caching) — Sprint 3 can add a materialized view or computed column
- Activity `type` is a free-form string, not an enum — gives flexibility but no DB-level validation
- No E2E/integration tests yet — all 265 tests are unit tests with mocked Prisma
- AI generation is synchronous (no streaming or queue) — could timeout for slow responses
- Document versioning stores full content (no diffs) — could grow large over time

Scoring Rubric Quick Reference

Category	Points	Demo Coverage
Preparation and Time Discipline	8	Pre-demo checklist, pacing, finish within 15 min
Dashboard search/filter/sort + status	6	Section A1
Job Detail workflow	6	Section A2
Interview/follow-up/timeline/outcome	6	Section A3
Profile + AI docs + save/link + metrics	6	Section A4
Workflow/data-integrity proof	5	Section B1 — stage transition transaction + metrics
CI pipeline evidence	3	Section B2 — GitHub Actions
Meaningful unit tests	5	Section B2 — specific tests shown
Architecture understanding	5	Section B3 — Q&A
Total	50	

Failure Mode Checklist

- ☐ All core Sprint 2 workflows demonstrated (A1–A4)
- ☐ Workflow/data-integrity proof shown (B1) — stage transition transaction
- ☐ CI green and viewable (B2)
- ☐ At least one non-happy-path test explained (B2)
- ☐ Completed within 15-minute slot